

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НОВОСИБИРСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
Факультет информационных технологий
Кафедра параллельных вычислений**

ОТЧЕТ

О ВЫПОЛНЕНИИ ЛАБОРАТОРНОЙ РАБОТЫ №1

«ОПРЕДЕЛЕНИЕ ВРЕМЕНИ РАБОТЫ ПРИКЛАДНЫХ ПРОГРАММ»

студента 2 курса, 24204 группы

Агапченко Владислава Вадимовича

Направление 09.03.01 – «Информатика и вычислительная техника»

Преподаватель:
к.т.н. А.Ю. Власенко

Новосибирск 2025

СОДЕРЖАНИЕ

ЦЕЛЬ.....	3
ЗАДАНИЕ.....	3
ОПИСАНИЕ РАБОТЫ.....	4
1. Написание программы согласно варианту задания.....	4
2. Проверка корректности программы.....	4
3. Подбор значения параметра N.....	5
4. Компиляция на разных уровнях оптимизации.....	5
5. Измерение времени работы программы.....	5
6. Измерение времени выполнения программы утилитой time.....	6
ЗАКЛЮЧЕНИЕ.....	8
Приложение 1. Код исходной программы <i>prog.cpp</i>	9
Приложение 2. Код файла сборки <i>makefile</i>	11
Приложение 3. Скрипт измерения времени <i>time.sh</i>	12
Приложение 4. Измерение времени работы при оптимизации <i>O0</i>	13
Приложение 5. Измерение времени работы при оптимизации <i>O1</i>	14
Приложение 6. Измерение времени работы при оптимизации <i>O2</i>	15
Приложение 7. Измерение времени работы при оптимизации <i>O3</i>	16
Приложение 8. Измерение времени работы при оптимизации <i>Os</i>	17
Приложение 9. Измерение времени работы при оптимизации <i>Ofast</i>	18
Приложение 10. Измерение времени работы при оптимизации <i>Og</i>	19

ЦЕЛЬ

Научиться измерять время работы программы и подпрограммы с высокой точностью, определять подходящую методику измерения времени выполнения программы.

Увидеть различия во времени выполнения при компиляции программы с разными уровнями оптимизации.

ЗАДАНИЕ

1. Написать программу на языке C или C++, которая реализует алгоритм вычисления числа π с помощью разложения в ряд (ряд Грегори-Лейбница) по формуле Лейбница N первых членов ряда:

$$\pi = 4 * \sum_{n=0}^{\infty} \frac{(-1)^n}{2n+1} = 4 - \frac{4}{3} + \frac{4}{5} - \frac{4}{7} + \dots + 4 * \frac{(-1)^n}{(2n+1)} + \dots$$

2. Проверить правильность работы программы на нескольких тестовых наборах входных данных.
3. Выбрать значение параметра N таким, чтобы время работы программы было порядка от 20 до 40 секунд.
4. Программу скомпилировать компилятором GCC с уровнями оптимизации -O0, -O1, -O2, -O3, -Os, -Ofast, -Og под архитектуру процессора x86 (x86-64).
5. Для каждого из семи вариантов компиляции измерить время работы программы при нескольких значениях N (как минимум: 0.5* N, N, 1.5* N).
6. Продемонстрировать владение утилитой time (как минимум один запуск).
7. Составить отчет по практической работе.

ОПИСАНИЕ РАБОТЫ

1. Написание программы согласно варианту задания

Для выполнения задания была написана программа prog.cpp на языке C++, код программы представлен в приложении 1.

Программа принимает в качестве первого аргумента число N – количество членов ряда Грегори-Лейбница, в качестве второго опционального аргумента – число итераций с запуском подпрограммы в цикле.

2. Проверка корректности программы

Код программы был скомпилирован и проверен при разных входных данных. В листинге 1 представлена работа программы при разных входных данных.

Листинг 1 – Компилирование и проверка работы prog.cpp

```
01 evm035@comrade:~/pr1$ g++ ./prog.cpp -o prog
02 evm035@comrade:~/pr1$ ./prog 1000
03 Leibniz precision = 1000
04 Calculated PI = 3.14059265383979413
05 Calculation took 0 sec.
06 evm035@comrade:~/pr1$ ./prog 1000000
07 Leibniz precision = 1000000
08 Calculated PI = 3.14159165358977432
09 Calculation took 0.01 sec.
10 evm035@comrade:~/pr1$ ./prog 1000000000
11 Leibniz precision = 1000000000
12 Calculated PI = 3.14159265258805043
13 Calculation took 9.03 sec.
14 evm035@comrade:~/pr1$ ./prog 2000000000
15 Leibniz precision = 2000000000
16 Calculated PI = 3.14159265308807667
17 Calculation took 17.82 sec.
```

В таблице 1 представлены результаты вычисления числа π .

Входной параметр N	Результат работы программы
1000	3.14059265383979413
1000000	3.14159165358977432
1000000000	3.14159265258805043
1500000000	3.1415926529214242

Таблица 1 – Результаты вычисления числа π в prog.cpp

С увеличением параметра N, точность результирующего значения числа π повышается, приближаясь к значению:

$$\pi = 3,1415926535897932384626433832795...$$

Следовательно, программа работает корректно.

3. Подбор значения параметра N

Измерение времени работы функции `leibniz`, вычисляющей значение числа π , измеряется библиотечной функцией `times`. Функция `times` определяет, сколько квантов времени работы процессора проводились вычисления, а затем число квантов переводится во время выполнения.

Было подобрано значение $N=3 \cdot 10^9$, при котором время работы программы составляет ~ 26 секунд. В листинге 2 представлен запуск программы с подобранным значением N.

Листинг 2 - Время работы программы `prog.cpp`

```
01 evm035@comrade:~/pr1$ ./prog 3000000000 5
02 Leibniz precision = 3000000000
03 Calculated PI = 3.14159265325492543
04 Calculation took 27.43 sec.
05 Calculated PI = 3.14159265325492543
06 Calculation took 26.61 sec.
07 Calculated PI = 3.14159265325492543
08 Calculation took 26.6 sec.
09 Calculated PI = 3.14159265325492543
10 Calculation took 26.35 sec.
11 Calculated PI = 3.14159265325492543
12 Calculation took 27.29 sec.
13 Average calculation time: 26.856 sec.
```

4. Компиляция на разных уровнях оптимизации

Для компиляции программы на разных уровнях оптимизации был написан `makefile`, код представлен в приложении 2. В листинге 3 представлен процесс компиляции программы утилитой `make`.

Листинг 3 – Компиляция `prog.cpp` на разных уровнях оптимизации

```
01 evm035@comrade:~/pr1$ make
02 g++ -Wall -std=c++17 -O0 -o prog-00 prog.cpp
03 g++ -Wall -std=c++17 -O1 -o prog-01 prog.cpp
04 g++ -Wall -std=c++17 -O2 -o prog-02 prog.cpp
05 g++ -Wall -std=c++17 -O3 -o prog-03 prog.cpp
06 g++ -Wall -std=c++17 -Os -o prog-0s prog.cpp
07 g++ -Wall -std=c++17 -Ofast -o prog-Ofast prog.cpp
08 g++ -Wall -std=c++17 -Og -o prog-0g prog.cpp
```

5. Измерение времени работы программы

Для измерения времени был написан скрипт `time.sh`, который первым аргументом принимает уровень оптимизации, вторым - параметр N, третьим – число итераций с запуском подпрограммы в цикле.

Скрипт измеряет время выполнения программы для набора входных данных $0.5*N$, N , $1.5*N$. Код скрипта представлен в приложении 3.

Результаты вызовов скрипта для разных уровней оптимизаций представлены в приложениях 4-10.

На графике 1 изображено различие во времени выполнения подпрограммы между разными уровнями оптимизации в зависимости от входных данных N .

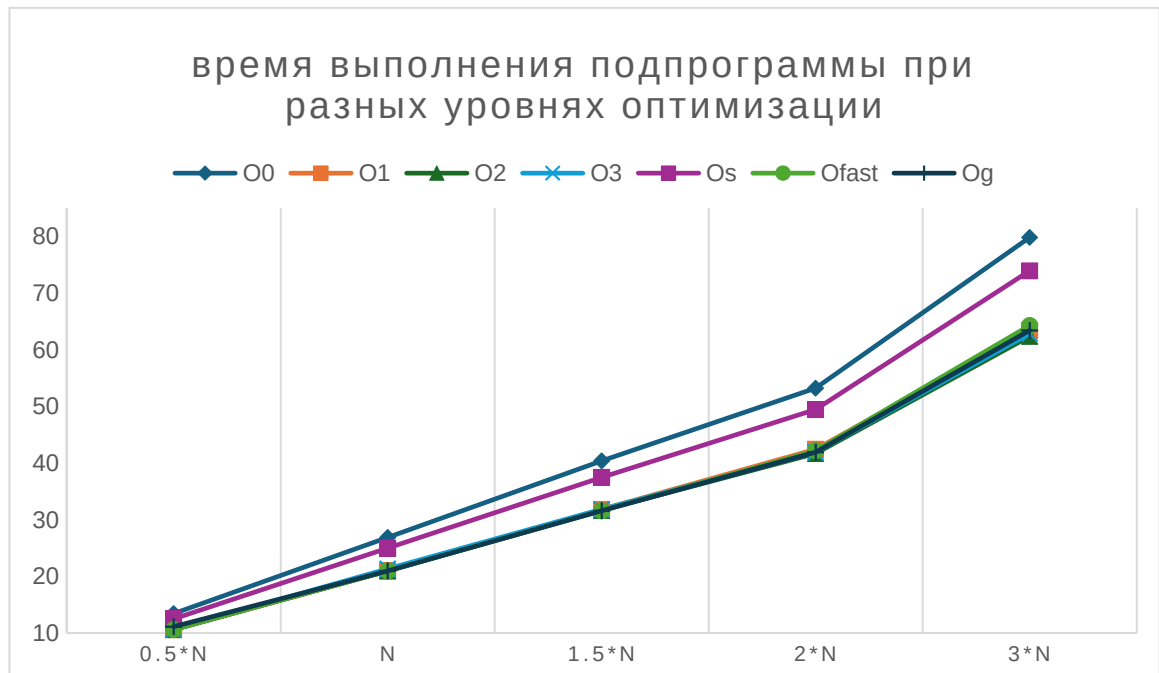


График 1 – Время выполнения подпрограммы при разных уровнях оптимизации

На графике заметно, что наименее эффективным вариантом является уровень оптимизации O0 (компиляция без оптимизации). Ввиду простоты программы, время выполнения программы при уровнях оптимизации O1, O2, O3, Ofast практически идентично.

6. Измерение времени выполнения программы утилитой time

Утилита time измеряет три вида время выполнения программы:

- real – полное время выполнения, от запуска до завершения программы, согласно системному таймеру;
- user – время выполнения пользовательского процесса;
- system – время, затраченное на выполнение системных вызовов программы.

В листинге 4 представлено измерение времени программы утилитой time.

Листинг 4 - Время работы программы prog.cpp с утилитой time

```
01 time ./prog-00 30000000000
02 Leibniz precision = 30000000000
03 Calculated PI = 3.141592653254925427
04 Calculation took 26.800 sec.
05
06 real      0m26,813s
07 user      0m26,806s
08 sys       0m0,000s
```

В таблице 2 представлены результаты измерения времени prog.cpp при $N=3 \cdot 10^9$ и 2-мя итерациями вычислений.

Вид времени	Измеренное время, сек.
real	26,813
user	26,806
sys	0,000

Таблица 2 – Измерение времени выполнения prog.cpp утилитой time

Таким образом, полное время работы программы составило 26,813 секунды, из полного времени выполнения на выполнение процесса самой программы было затрачено 26,806 секунд. Так как программа обращается к операционной системе только для вывода в консоль, то на системные вызовы времени затрачено практически не было.

ЗАКЛЮЧЕНИЕ

В результате выполнения лабораторной работы, мной были изучены способы измерения времени выполнения работы программы (утилита `time`) и подпрограммы (функция `times`).

Были проведены измерения времени выполнения работы подпрограммы при разных уровнях оптимизации и начертан график для наглядного изображения отличий времени выполнения.

Приложение 1. Код исходной программы prog.cpp

```
01 #include <cstdlib>
02 #include <iostream>
03 #include <stdexcept>
04 #include <sys/times.h>
05 #include <unistd.h>
06
07 double leibniz(long long n) {
08     double pi = 0;
09
10     for (long long k = 0, sign = 1; k < n; k++) {
11         pi += (double) sign / (2 * k + 1);
12         sign = -sign;
13     }
14
15     return 4 * pi;
16 }
17
18 int main(int argc, char** argv) {
19     struct tms start, end;
20     long clocks_per_sec = sysconf(_SC_CLK_TCK);
21     long clocks;
22
23     int runs = 1;
24     double pi;
25     long long n;
26
27     if (argc < 2) {
28         std::cout << "Missing argument N [long long]" <<
29         std::endl;
30         return EXIT_FAILURE;
31     }
32
33     try {
34         n = std::stoll(argv[1]);
35         if (argc == 3) {
36             runs = std::stoi(argv[2]);
37         }
38     } catch (const std::invalid_argument& e) {
39         std::cout << "Invalid argument: " << e.what() <<
40         std::endl;
41         return EXIT_FAILURE;
42     } catch (const std::out_of_range& e) {
43         std::cout << "Out of range: " << e.what() << std::endl;
44         return EXIT_FAILURE;
45     }
46
47     double total_time = 0;
48
49     std::cout << "Leibniz precision = " << n << std::endl;
```

```
50  for (int i = 0; i < runs; i++) {
51      times(&start);
52      pi = leibniz(n);
53      times(&end);
54
55      clocks = end.tms_utime - start.tms_utime;
56      double time = (double) clocks / clocks_per_sec;
57      total_time += time;
58
59      printf("Calculated PI = %.19g\n", pi);
60      printf("Calculation took %.5g sec.\n", time);
61  }
62
63  if (runs > 1) {
64      std::cout << "Average calculation time: " <<
65          total_time / runs << " sec." << std::endl;
66  }
67
68  return EXIT_SUCCESS;
69 }
```

Приложение 2. Код файла сборки *makefile*

```
01 CC = g++
02 CFLAGS = -Wall -std=c++17
03 SRC = prog.cpp
04 TARGET_BASE = prog
05 OPT_LEVELS = -O0 -O1 -O2 -O3 -Os -Ofast -Og
06
07 TARGETS = $(foreach opt, $(OPT_LEVELS),
08   $(patsubst -%,%, $(opt)))
09
10 all: $(TARGETS)
11
12 0%: $(SRC)
13   $(CC) $(CFLAGS) -O$* -o $(TARGET_BASE)-$@ $<
14
15 clean:
16   rm -f $(foreach opt, $(TARGETS), $(TARGET_BASE)-$(opt))
17
18 .PHONY: all clean
```

Приложение 3. Скрипт измерения времени *time.sh*

```
01 #!/bin/bash
02
03 OPT="00"
04 N0=1000000000
05 RUNS=1
06
07 if [[ -n "$1" ]]; then
08     OPT="$1"
09 fi
10
11 if [[ -n "$2" ]]; then
12     N0="$2"
13 fi
14
15 if [[ -n "$3" ]]; then
16     RUNS="$3"
17 fi
18
19 N_VALUES=(
20     $((N0 / 2))
21     $((N0))
22     $((N0 * 3/2))
23     $((N0 * 2))
24     $((N0 * 3))
25 )
26
27 printf "==== Time test =====\n"
28 for N in "${N_VALUES[@]}; do
29     printf "| N = %22.0f |\n" "$N"
30     printf "=====\n"
31     "./prog-$OPT" "$N" "$RUNS"
32     printf "=====\n"
33 done
```

Приложение 4. Измерение времени работы при оптимизации

00

```
01 evm035@comrade:~/pr1$ ./time.sh 00 3000000000
02 ===== Time test =====
03 | N =                1500000000 |
04 =====
05 Leibniz precision = 1500000000
06 Calculated PI = 3.1415926529214242
07 Calculation took 13.43 sec.
08 =====
09 | N =                3000000000 |
10 =====
11 Leibniz precision = 3000000000
12 Calculated PI = 3.141592653254925427
13 Calculation took 26.86 sec.
14 =====
15 | N =                4500000000 |
16 =====
17 Leibniz precision = 4500000000
18 Calculated PI = 3.141592653365865129
19 Calculation took 40.37 sec.
20 =====
21 | N =                6000000000 |
22 =====
23 Leibniz precision = 6000000000
24 Calculated PI = 3.141592653421574788
25 Calculation took 53.2 sec.
26 =====
27 | N =                9000000000 |
28 =====
29 Leibniz precision = 9000000000
30 Calculated PI = 3.141592653477197405
31 Calculation took 79.8 sec.
32 =====
```

Приложение 5. Измерение времени работы при оптимизации

O1

```
01 evm035@comrade:~/pr1$ ./time.sh O1 3000000000
02 ===== Time test =====
03 | N =                1500000000 |
04 =====
05 Leibniz precision = 1500000000
06 Calculated PI = 3.1415926529214242
07 Calculation took 10.59 sec.
08 =====
09 | N =                3000000000 |
10 =====
11 Leibniz precision = 3000000000
12 Calculated PI = 3.141592653254925427
13 Calculation took 20.95 sec.
14 =====
15 | N =                4500000000 |
16 =====
17 Leibniz precision = 4500000000
18 Calculated PI = 3.141592653365865129
19 Calculation took 31.8 sec.
20 =====
21 | N =                6000000000 |
22 =====
23 Leibniz precision = 6000000000
24 Calculated PI = 3.141592653421574788
25 Calculation took 42.42 sec.
26 =====
27 | N =                9000000000 |
28 =====
29 Leibniz precision = 9000000000
30 Calculated PI = 3.141592653477197405
31 Calculation took 63.38 sec.
32 =====
```

Приложение 6. Измерение времени работы при оптимизации O2

```
01 evm035@comrade:~/pr1$ ./time.sh O2 3000000000
02 ===== Time test =====
03 | N =                1500000000 |
04 =====
05 Leibniz precision = 1500000000
06 Calculated PI = 3.1415926529214242
07 Calculation took 10.6 sec.
08 =====
09 | N =                3000000000 |
10 =====
11 Leibniz precision = 3000000000
12 Calculated PI = 3.141592653254925427
13 Calculation took 20.95 sec.
14 =====
15 | N =                4500000000 |
16 =====
17 Leibniz precision = 4500000000
18 Calculated PI = 3.141592653365865129
19 Calculation took 31.64 sec.
20 =====
21 | N =                6000000000 |
22 =====
23 Leibniz precision = 6000000000
24 Calculated PI = 3.141592653421574788
25 Calculation took 41.65 sec.
26 =====
27 | N =                9000000000 |
28 =====
29 Leibniz precision = 9000000000
30 Calculated PI = 3.141592653477197405
31 Calculation took 62.34 sec.
32 =====
```

Приложение 7. Измерение времени работы при оптимизации ОЗ

```
01 evm035@comrade:~/pr1$ ./time.sh ОЗ 3000000000
02 ===== Time test =====
03 | N =                1500000000 |
04 =====
05 Leibniz precision = 1500000000
06 Calculated PI = 3.1415926529214242
07 Calculation took 10.6 sec.
08 =====
09 | N =                3000000000 |
10 =====
11 Leibniz precision = 3000000000
12 Calculated PI = 3.141592653254925427
13 Calculation took 21.34 sec.
14 =====
15 | N =                4500000000 |
16 =====
17 Leibniz precision = 4500000000
18 Calculated PI = 3.141592653365865129
19 Calculation took 31.81 sec.
20 =====
21 | N =                6000000000 |
22 =====
23 Leibniz precision = 6000000000
24 Calculated PI = 3.141592653421574788
25 Calculation took 41.9 sec.
26 =====
27 | N =                9000000000 |
28 =====
29 Leibniz precision = 9000000000
30 Calculated PI = 3.141592653477197405
31 Calculation took 62.81 sec.
32 =====
```

Приложение 8. Измерение времени работы при оптимизации Os

```
01 evm035@comrade:~/pr1$ ./time.sh Os 3000000000
02 ===== Time test =====
03 | N =                1500000000 |
04 =====
05 Leibniz precision = 1500000000
06 Calculated PI = 3.1415926529214242
07 Calculation took 12.48 sec.
08 =====
09 | N =                3000000000 |
10 =====
11 Leibniz precision = 3000000000
12 Calculated PI = 3.141592653254925427
13 Calculation took 24.98 sec.
14 =====
15 | N =                4500000000 |
16 =====
17 Leibniz precision = 4500000000
18 Calculated PI = 3.141592653365865129
19 Calculation took 37.45 sec.
20 =====
21 | N =                6000000000 |
22 =====
23 Leibniz precision = 6000000000
24 Calculated PI = 3.141592653421574788
25 Calculation took 49.43 sec.
26 =====
27 | N =                9000000000 |
28 =====
29 Leibniz precision = 9000000000
30 Calculated PI = 3.141592653477197405
31 Calculation took 73.89 sec.
32 =====
```

33

Приложение 9. Измерение времени работы при оптимизации Ofast

```
01 evm035@comrade:~/pr1$ ./time.sh Ofast 3000000000
02 ===== Time test =====
03 | N =                1500000000 |
04 =====
05 Leibniz precision = 1500000000
06 Calculated PI = 3.1415926529214242
07 Calculation took 10.6 sec.
08 =====
09 | N =                3000000000 |
10 =====
11 Leibniz precision = 3000000000
12 Calculated PI = 3.141592653254925427
13 Calculation took 20.95 sec.
14 =====
15 | N =                4500000000 |
16 =====
17 Leibniz precision = 4500000000
18 Calculated PI = 3.141592653365865129
19 Calculation took 31.56 sec.
20 =====
21 | N =                6000000000 |
22 =====
23 Leibniz precision = 6000000000
24 Calculated PI = 3.141592653421574788
25 Calculation took 42.16 sec.
26 =====
27 | N =                9000000000 |
28 =====
29 Leibniz precision = 9000000000
30 Calculated PI = 3.141592653477197405
31 Calculation took 64.25 sec.
32 =====
```

Приложение 10. Измерение времени работы при оптимизации

Og

```
01 evm035@comrade:~/pr1$ ./time.sh Og 3000000000
02 ===== Time test =====
03 | N =                1500000000 |
04 =====
05 Leibniz precision = 1500000000
06 Calculated PI = 3.1415926529214242
07 Calculation took 11.12 sec.
08 =====
09 | N =                3000000000 |
10 =====
11 Leibniz precision = 3000000000
12 Calculated PI = 3.141592653254925427
13 Calculation took 20.95 sec.
14 =====
15 | N =                4500000000 |
16 =====
17 Leibniz precision = 4500000000
18 Calculated PI = 3.141592653365865129
19 Calculation took 31.56 sec.
20 =====
21 | N =                6000000000 |
22 =====
23 Leibniz precision = 6000000000
24 Calculated PI = 3.141592653421574788
25 Calculation took 41.9 sec.
26 =====
27 | N =                9000000000 |
28 =====
29 Leibniz precision = 9000000000
30 Calculated PI = 3.141592653477197405
31 Calculation took 63.37 sec.
32 =====
```